

---

## UNIT – 3

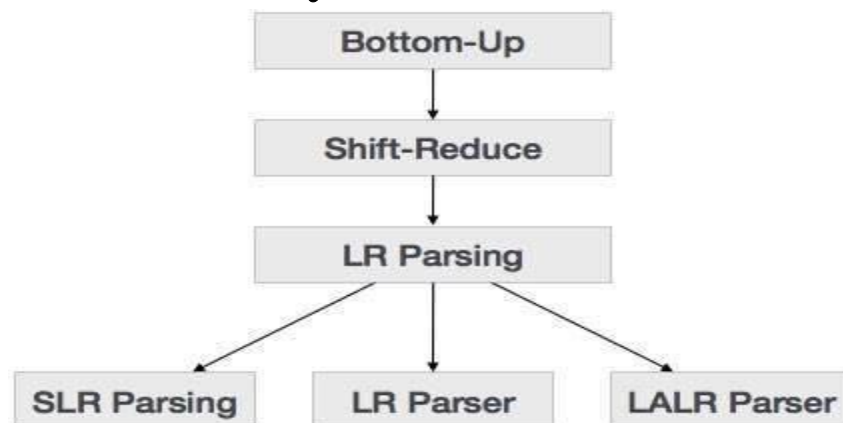
**I. Bottom up Parsing:** Shift-reduce parser, Operator precedence parser, LR parsers.

**II. Intermediate Code Generation:** Intermediate Language, Declarations, Assignment statements, Boolean expressions, Case statement, Backpatching , Procedure call.

**III. Error detection and recovery:** Lexical phase errors, Syntactic phase errors, Semantic errors.

---

### **I. BOTTOM-UP PARSING TECHNIQUES**



- A bottom-up parsing corresponds to the construction of a parse tree for an input string beginning at the leaves (the bottom) and working up towards the root (the top).
- Bottom-up parsers generally operate by choosing, on the basis of the next input symbol (look-ahead symbol) and the contents of the stack, whether to shift the next input onto the stack, or to reduce some symbols at the top of the stack.
- A reduce step takes a production body at the top of the stack and replaces it by the head of the production

#### **3.1 Shift Reduce Parsing (SR parser)**

- Shift – Reduce Parsing is a form of bottom-up parsing in which a stack holds the grammar symbols and an input buffer holds the rest of the string to be parsed.
- It uses \$ to mark the bottom of the stack and also the right end of the input.
- There are four possible actions a Shift-Reduce parser can make,
  1. **SHIFT** – Shift the next input symbol onto the top of the stack.
  2. **REDUCE** – The right end of the handle must be at the top of the stack, locate the left end of the handle within the stack and decide with what non-terminal to replace the handle.
  3. **ACCEPT** - Announce the successful completion of parsing
  4. **ERROR** - Discover a syntax error and call an error recovery routine.

Example:

G :  $E \rightarrow E + T \mid T$        $T \rightarrow T * F \mid F$        $F \rightarrow (E) \mid id$

Input string: id \* id

Stack implementation of SR Parser.

| Stack     | Buffer     | Handle | Action                          |
|-----------|------------|--------|---------------------------------|
| \$        | id * id \$ |        | Shift                           |
| \$ id     | * id \$    | id     | Reduce by $F \rightarrow id$    |
| \$ F      | * id \$    | F      | Reduce by $T \rightarrow F$     |
| \$ T      | * id \$    |        | Shift                           |
| \$ T *    | id \$      |        | Shift                           |
| \$ T * id | \$         | id     | Reduce by $F \rightarrow id$    |
| \$ T * F  | \$         | F      | Reduce by $T \rightarrow T * F$ |
| \$ T      | \$         | T      | Reduce by $E \rightarrow T$     |
| \$ E      | \$         |        | Accept                          |

### **3.2 OR PARSER – OPERATOR PRECEDENCE PARSER**

**Operator Grammar:** An Operator Grammar is a grammar with the property that

- (i) No production has  $\epsilon$  on its right side
- (ii) No production has two adjacent Non-terminals

**Operator Precedence Grammar:**

- An Operator precedence grammar is an operator grammar with the disjoint relation (includes  $<$ ,  $>$  and  $=$ ) between any pair of terminals
- There are three precedence relations exists between any pair of terminals a and b as
  - $a < \cdot b$ , a gives precedence to b
  - $a \cdot > b$ , a takes precedence over b
  - $a \doteq b$ , a has equal precedence as b

#### **OR Parser – Steps**

1. Check for Operator Grammar
2. Computation of LEADING

3. Computation of TRAILING
4. Computation of PRECEDENCE RELATION
5. Construction of Precedence Table
6. OR Parsing

#### Computation of LEADING – Algorithm

**(Rule 1)** 'a' is in LEADING (A) if,  $A \rightarrow aY$  or  $A \rightarrow XaY$  where X, Y are grammar symbols.

**(Rule 2)** If  $A \rightarrow BX$  is a production, then  
Include every terminal in LEADING (B) to LAEADING (A).

#### Computation of TRAILING – Algorithm

**(Rule 1)** 'a' is in TRAILING (A) if,  $A \rightarrow Xa$  or  $A \rightarrow XaY$  where X, Y are grammar symbols.

**(Rule 2)** If  $A \rightarrow XB$  is a production, then  
Include every terminal in TRAILING (B) to TRAILING (A).

#### Computation of PRECEDENCE RELATION – Algorithm

- (Rule 1)** Let S is a start symbol of the grammar,  
Set  $\$ \prec a$ , for all symbols 'a' in LEADING(S)  
Set  $b \succ \$$ , for all symbols 'b' in TRAILING(S).
- (Rule 2)** If  $A \rightarrow XY$  or  $A \rightarrow XBY$ , where X and Y are terminals, then  
Set  $X = Y$ .
- (Rule 3)** If  $A \rightarrow XB$  where X is a terminal, then  
Set  $X \prec a$ , for all symbols 'a' in LEADING(B)
- (Rule 4)** If  $A \rightarrow BX$  where X is a terminal, then  
Set  $b \succ X$ , for all symbols 'b' in TRAILING(B).

#### PRECEDENCE Parsing – Algorithm

- (Rule 1)** If Top of the Stack and current Input symbol is \$, then  
ACCEPT and announce successful completion of parsing.
- (Rule 2)** If Top = a, current Input symbol is b and  $a \prec b$  or  $a = b$ , then  
SHIFT b onto the stack.
- (Rule 3)** else If  $a \succ b$ , then REDUCE.  
Pop off all the symbols until the backward scan reaches the first  $\prec$  from top of the stack.
- (Rule 4)** Announce ERROR if undefined entry encountered.

**Example:** Please refer class-work note book.

### **3.3 LR PARSERS**

- The LR parser is a non-recursive, shift-reduce, bottom-up parser.
- It uses a wide class of context free grammar which makes it the most efficient syntax analysis technique.
- LR parsers are also known as **LR(k)** parsers,  
Where
  - L stands for left-to-right scanning of the input stream;
  - R stands for the construction of right-most derivation in reverse, and
  - K denotes the number of look-ahead symbols to make decisions.
- There are three widely used algorithms available for constructing an LR parser:
  1. **SLR(1) – Simple LR Parser:**
    - Works on smallest class of grammar
    - Few number of states, hence very small table
    - Simple and fast construction
  2. **CLR(1) – Canonical LR Parser:**
    - Works on complete set of LR(1) Grammar
    - Generates large table and large number of states
    - Slow construction
  3. **LALR(1) – Look-Ahead LR Parser:**
    - Works on intermediate size of grammar
    - Number of states are same as in SLR(1)

**Organization of LR Parsers:** The LR Parser consists of

**Input Buffer – to hold the input string to be parsed**

**Stack – holds a sequence of states of LR Items set and the grammar symbols.**

**Driver Program – is the same for all LR Parsers. Only the parsing table changes from one parser to another.**

**Parsing table:** The parsing table consists of two parts:

- (1) a parsing function ACTION and
- (2) a goto function GOTO.

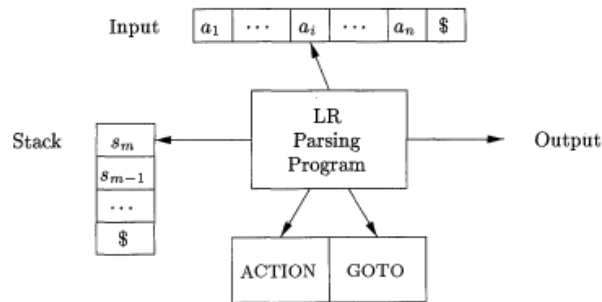


Figure 4.35: Model of an LR parser

1. The ACTION function takes as arguments a state  $i$  and a terminal  $a$  (or  $\$,$  the input end marker). The value of ACTION[ $i, a$ ] can have one of four forms:
  - a. Shift  $j$ , where  $j$  is a state. The action taken by the parser effectively shifts input  $a$  to the stack, but uses state  $j$  to represent  $a$ .
  - b. Reduce  $A \rightarrow \beta$ . The action of the parser effectively reduces  $\beta$  on the top of the stack to head  $A$ .
  - c. Accept. The parser accepts the input and finishes parsing.
  - d. Error. The parser discovers an error in its input and takes some corrective action.
2. Extend the GOTO function, defined on sets of items, to states: if GOTO[ $I_i, A$ ] =  $I_j$ , then GOTO also maps a state  $i$  and a non terminal  $A$  to state  $j$ .

### SLR – Simple LR Parser

- SLR Grammar: A grammar for which an SLR parser can be constructed is said to be SLR Grammar.
- Steps to construct the SLR Parser
  1. Computation of LR (0) Items for the given grammar  $G$
  2. Construction of SLR Parsing table using LR (0) items
  3. LR Parsing the Input String

### LR (0) Item

- An LR Item of a grammar  $G$  is a production of  $G$  with a DOT at some position of the RHS.
  - (1) A production  $A \rightarrow XYZ$  yields the four items
 
$$A \rightarrow .XYZ$$

$$A \rightarrow X.YZ$$

$$A \rightarrow XY.Z$$

$$A \rightarrow XYZ.$$
  - (2) A Production  $A \rightarrow \epsilon$  generates only one item as  $A \rightarrow .$
- LR (0) Items provides basis for constructing SLR Parser.

### LR Items – Classification

The LR Items can be classified into two different classes as

**Kernel Items** – It include the Initial Item  $S' \rightarrow S$  and all other items whose dots are not at the leftmost end.

**Non-Kernel Items** - It include items which have their dots at their leftmost end.

### Computation of LR (0) Items – Steps

LR (0) Items can be computed using three steps

1. Augmented Grammar
2. Closure Function
3. GOTO Function

**(i) Augmented Grammar:** If G is a grammar with the start symbol S, then the augmented grammar  $G'$  for G is formed with new start symbol  $S'$  and production  $S' \rightarrow S$ .

Example:

G:  $E \rightarrow E + E \mid id$

The augmented grammar

$G'$ :  $E' \rightarrow E$   
 $E \rightarrow E + E \mid id$

### Significance of Augmented Grammar:

- An Augmented Grammar indicates to the parser, when it should stop parsing and announce the acceptance of the inputs.
- The parsing process is completed when a parser is about to reduce  $S' \rightarrow S$ .

### (ii) CLOSURE Function:

Let I is a set of Items for a grammar G, then Closure (I) is constructed from I by using two rules, Initially every Item in I is added to CLOSURE (I)

If  $A \rightarrow \alpha.B\beta$  is in CLOSURE (I) and  $B \rightarrow \gamma$  is a production then  
Add item  $B \rightarrow \gamma$  into I, if it is not there.

### (iii) GOTO Function:

The function GOTO (I, X), where X is a Grammar Symbol,

If  $A \rightarrow \alpha.X\beta$  is in I then GOTO (I, X) defined as the CLOSURE ( $A \rightarrow \alpha X \beta$ )

## 2. Construction of SLR Parsing table – Algorithm

Let  $C = \{I_0, I_1, I_2, \dots, I_n\}$  be the collection of LR (0) Items for  $G'$  and Let  $I_j$  is a set in C,

(R1) If  $\text{GOTO} (I_j, a) = I_k$  then set  $\text{ACTION} [j, a] = \text{SHIFT } K \text{ (or) } S_k$

(R2) If  $\text{GOTO} (I_j, A) = I_k$  then set  $\text{GOTO} [j, A] = K$

(R3) If  $A \rightarrow \alpha$  is in set  $I_j$  then set  $\text{ACTION} [j, a] = \text{REDUCE BY } A \rightarrow \alpha$  for every symbol 'a' in FOLLOW (A)

- (R4) If  $S' \rightarrow S$  is in set  $I_j$  then set ACTION  $[j, \$] = \text{ACCEPT}$   
 (R5) All the undefined entries are ERROR.

*Example: Construct the Simple LR Parsing table for the following grammar  $S \rightarrow CC$ ,  $C \rightarrow eC$ ,  $C \rightarrow d$ . And parse the string eded.*

Given:

G:  $S \rightarrow CC$   
 $C \rightarrow eC$   
 $C \rightarrow d$

GOTO ( $I_0, d$ )  
 $C \rightarrow d. \dots\dots(I_4)$

Computation of LR (0) Items

1. Augmented Grammar

G':  $S' \rightarrow S$   
 $S \rightarrow CC$   
 $C \rightarrow eC$   
 $C \rightarrow d$

GOTO ( $I_2, C$ )  
 $S \rightarrow CC. \dots\dots(I_5)$

2. CLOSURE ( $S' \rightarrow S$ )

$S' \rightarrow .S$   
 $S \rightarrow .CC$   
 $C \rightarrow .eC$   
 $C \rightarrow .d \dots\dots(I_0)$

GOTO ( $I_2, e$ )  
 $C \rightarrow e.C$   
 $C \rightarrow .eC$   
 $C \rightarrow .d \dots\dots(I_3)$

GOTO ( $I_2, d$ )  
 $C \rightarrow d. \dots\dots(I_4)$

3. GOTO ( $I_0, S$ )

$S' \rightarrow S. \dots\dots(I_1)$

GOTO ( $I_3, C$ )  
 $C \rightarrow eC. \dots\dots(I_6)$

GOTO ( $I_0, C$ )

$S \rightarrow C.C$   
 $C \rightarrow .eC$   
 $C \rightarrow .d \dots\dots(I_2)$

GOTO ( $I_3, e$ )  
 $C \rightarrow e.C$   
 $C \rightarrow .eC$   
 $C \rightarrow .d \dots\dots(I_3)$

GOTO ( $I_0, e$ )

$C \rightarrow e.C$   
 $C \rightarrow .eC$   
 $C \rightarrow .d \dots\dots(I_3)$

GOTO ( $I_3, d$ )  
 $C \rightarrow d. \dots\dots(I_4)$

Construction of SLR Parsing Table

| State | ACTION |       |    | GOTO |   |
|-------|--------|-------|----|------|---|
|       | e      | d     | \$ | S    | C |
| 0     | $S_3$  | $S_4$ |    | 1    | 2 |

|   |                |                |                |  |   |
|---|----------------|----------------|----------------|--|---|
| 1 |                |                | ACC            |  |   |
| 2 | S <sub>3</sub> | S <sub>4</sub> |                |  | 5 |
| 3 | S <sub>3</sub> | S <sub>4</sub> |                |  | 6 |
| 4 | R <sub>3</sub> | R <sub>3</sub> | R <sub>3</sub> |  |   |
| 5 | R <sub>1</sub> | R <sub>1</sub> | R <sub>1</sub> |  |   |
| 6 | R <sub>2</sub> | R <sub>2</sub> | R <sub>2</sub> |  |   |

### SLR Parsing

| No | STACK           | BUFFER  | ACTION                                          |
|----|-----------------|---------|-------------------------------------------------|
| 1  | 0               | eded \$ | ACTION [0,e] = S <sub>3</sub> Shift             |
| 2  | 0e3             | ded \$  | ACTION [3,d] = S <sub>4</sub> Shift             |
| 3  | 0e3 <u>d4</u>   | ed \$   | ACTION [4,e] = R <sub>3</sub> Reduce by C → d   |
|    | 0e3C6           | ed \$   | Pop-off d4, Push C. GOTO(3,C)=6. Push 6         |
| 4  | 0 <u>e3C6</u>   | ed \$   | ACTION [6,e] = R <sub>2</sub> Reduce by C → eC  |
|    | 0C2             | ed \$   | Pop-off e3C6, Push C. GOTO(0,C)=2. Push 2       |
| 5  | 0C2             | ed \$   | ACTION [2,e] = S <sub>3</sub> Shift             |
| 6  | 0C2e3           | d \$    | ACTION [3,d] = S <sub>4</sub> Shift             |
| 7  | 0C2e3 <u>d4</u> | \$      | ACTION [4,\$] = R <sub>3</sub> Reduce by C → d  |
| 8  | 0C2 <u>e3C6</u> | \$      | ACTION [6,\$] = R <sub>2</sub> Reduce by C → eC |
| 9  | 0 <u>C2C5</u>   | \$      | ACTION [5,\$] = R <sub>1</sub> Reduce by S → CC |
| 10 | 0S1             | \$      | ACTION [1,\$] = Accept                          |

Problems on SLR Parsers – Please refer the Class Work Note-Book.

## CLR – Canonical LR Parser

- The CLR parser incorporates extra information in the state by redefining items to include a terminal symbol as a look-ahead as a second component.

- The general form of an Item is

$A \rightarrow \alpha.B\beta$  for SLR | LR (0) Item

becomes

$A \rightarrow \alpha.B\beta, a$

Where  $a$  is a terminal or a right-end marker  $\$,$  and the item is said to be LR (1) Item.

- Steps to construct the SLR Parser

1. Computation of LR (1) Items for the given grammar G
2. Construction of CLR Parsing table using LR (1) items
3. Parsing the Input String

### (ii) CLOSURE to compute LR(1) Item:

An Item of the form

$[A \rightarrow \alpha.B\beta, a]$  in the set I and  $B \rightarrow \gamma$  is a production it can be added into I as

$[A \rightarrow \alpha.B\beta, a]$

$[B \rightarrow \gamma, b]$

Where  $b = \text{FIRST}(\beta a).$

*Example: Construct the canonical LR Parsing table for the following grammar*

$S \rightarrow CC, C \rightarrow eC, C \rightarrow d.$

### Computation of LR (1) Items

#### 1. Augmented Grammar

G':  $S' \rightarrow S$

$S \rightarrow CC$

$C \rightarrow eC$

$C \rightarrow d$

#### 2. CLOSURE ( $S' \rightarrow S$ )

$S' \rightarrow .S, \$$

$S \rightarrow .CC, \$$

$C \rightarrow .eC, e|d$

$C \rightarrow .d, e|d \dots\dots(I_0)$

#### 3. GOTO ( $I_0, S$ )

$S' \rightarrow S., \$ \dots\dots(I_1)$

#### GOTO ( $I_0, C$ )

$S \rightarrow C.C, \$$

$C \rightarrow .eC, \$$

$C \rightarrow .d, \$ \dots\dots(I_2)$

#### GOTO ( $I_0, e$ )

$C \rightarrow e.C, e|d$

$C \rightarrow .eC, e|d$

$C \rightarrow .d, e|d \dots\dots(I_3)$

#### GOTO ( $I_0, d$ )

$C \rightarrow d., , e|d \dots\dots(I_4)$

#### GOTO ( $I_2, C$ )

$S \rightarrow CC., \$ \dots\dots(I_5)$

#### GOTO ( $I_2, e$ )

$C \rightarrow e.C, \$$

$C \rightarrow .eC, \$$

$C \rightarrow .d, \$ \dots\dots(I_6)$

GOTO (I<sub>2</sub>,d)

C → d., \$ .....(I<sub>7</sub>)

C → d. , e|d .....(I<sub>4</sub>)

GOTO (I<sub>3</sub>,C)

C → eC. , e|d .....(I<sub>8</sub>)

GOTO (I<sub>6</sub>,C)

C → eC., \$ .....(I<sub>9</sub>)

GOTO (I<sub>3</sub>,e)

C → e.C, e|d

C → .eC, e|d

C → .d, e|d .....(I<sub>3</sub>)

GOTO (I<sub>6</sub>,e)

C → e.C, \$

C → .eC, \$

C → .d, \$ .....(I<sub>6</sub>)

GOTO (I<sub>3</sub>,d)

GOTO (I<sub>6</sub>,d)

C → d., \$ .....(I<sub>7</sub>)

Construction of CLR Parsing Table

| State | ACTION         |                |                | GOTO |   |
|-------|----------------|----------------|----------------|------|---|
|       | e              | d              | \$             | S    | C |
| 0     | S <sub>3</sub> | S <sub>4</sub> |                | 1    | 2 |
| 1     |                |                | ACC            |      |   |
| 2     | S <sub>6</sub> | S <sub>7</sub> |                |      | 5 |
| 3     | S <sub>3</sub> | S <sub>4</sub> |                |      | 8 |
| 4     | R <sub>3</sub> | R <sub>3</sub> |                |      |   |
| 5     |                |                | R <sub>1</sub> |      |   |
| 6     | S <sub>6</sub> | S <sub>7</sub> |                |      | 9 |
| 7     |                |                | R <sub>3</sub> |      |   |
| 8     | R <sub>2</sub> | R <sub>2</sub> |                |      |   |
| 9     |                |                | R <sub>2</sub> |      |   |

More Problems on CLR Parsers – Please refer the Class Work Note-Book.

### LALR – Look-ahead LR Parser

- From the LR (1) Items of  $G'$  of  $G$ ,
  - Take a pair of similar looking states; say  $I_j$  and  $I_k$  each of these states are differentiated only by the look-ahead symbols.
  - Replace  $I_j$  and  $I_k$  by  $I_{jk}$ , the union of  $I_j$  and  $I_k$
  - The Goto's on any symbol  $X$  to  $I_j$  or  $I_k$  from any other states now replaced with  $I_{jk}$ .

*Example: Construct the canonical LR Parsing table for the following grammar*

*$S \rightarrow CC, C \rightarrow eC, C \rightarrow d$ . And parse the string eed.*

From Collection of LR (1) Items computed in CLR parser

State  $I_3$  and  $I_6$  are differentiated only by their look-ahead symbols,

- (i)  $C \rightarrow e.C, e|d$   
 $C \rightarrow .eC, e|d$   
 $C \rightarrow .d, e|d \dots\dots(I_3)$   
and  
 $C \rightarrow e.C, \$$   
 $C \rightarrow .eC, \$$   
 $C \rightarrow .d, \$ \dots\dots(I_6)$   
becomes  
 $C \rightarrow e.C, \$|e|d$   
 $C \rightarrow .eC, \$|e|d$   
 $C \rightarrow .d, \$|e|d \dots\dots(I_{36})$
- (ii)  $C \rightarrow d., e|d \dots\dots(I_4)$   
and  
 $C \rightarrow d., \$ \dots\dots(I_7)$   
becomes  
 $C \rightarrow d., \$|e|d \dots\dots(I_{47})$
- (iii)  $C \rightarrow eC., e|d \dots\dots(I_8)$   
and  
 $C \rightarrow eC., \$ \dots\dots(I_9)$   
becomes  
 $C \rightarrow eC., \$|e|d \dots\dots(I_{89})$

Construction of LALR Parsing Table

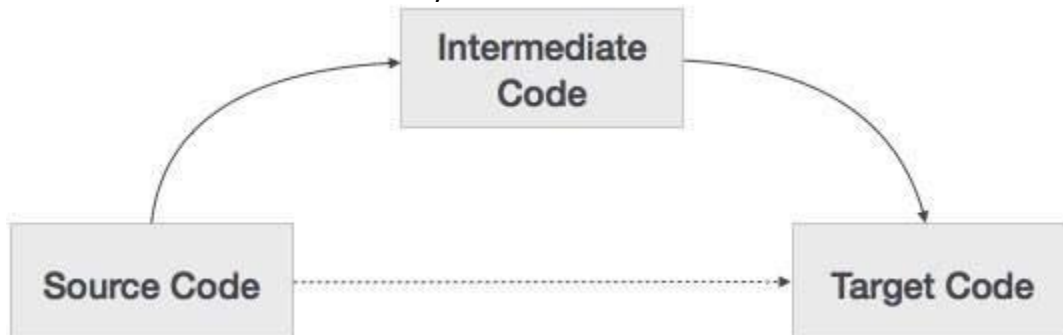
| State | ACTION          |                 |                | GOTO |    |
|-------|-----------------|-----------------|----------------|------|----|
|       | e               | d               | \$             | S    | C  |
| 0     | S <sub>36</sub> | S <sub>47</sub> |                | 1    | 2  |
| 1     |                 |                 | ACC            |      |    |
| 2     | S <sub>36</sub> | S <sub>47</sub> |                |      | 5  |
| 36    | S <sub>36</sub> | S <sub>47</sub> |                |      | 89 |
| 47    | R <sub>3</sub>  | R <sub>3</sub>  | R <sub>3</sub> |      |    |
| 5     |                 |                 | R <sub>1</sub> |      |    |
| 89    | R <sub>2</sub>  | R <sub>2</sub>  | R <sub>2</sub> |      |    |

LALR Parsing

| No | STACK              | BUFFER | ACTION                                           |
|----|--------------------|--------|--------------------------------------------------|
| 1  | 0                  | eed \$ | ACTION [0,e] = S <sub>36</sub> Shift             |
| 2  | 0e36               | ed \$  | ACTION [36,e] = S <sub>36</sub> Shift            |
| 3  | 0e36e36            | d \$   | ACTION [36,d] = S <sub>47</sub> Shift            |
| 4  | 0e36e36 <u>d47</u> | \$     | ACTION [47,\$] = R <sub>3</sub> Reduce by C → d  |
| 5  | 0e36 <u>e36C89</u> | \$     | ACTION [89,\$] = R <sub>2</sub> Reduce by C → eC |
| 6  | 0 <u>e36C89</u>    | \$     | ACTION [89,\$] = R <sub>2</sub> Reduce by C → eC |
| 7  | 0C2                | \$     | ACTION [2,\$] = Undefined, Error                 |

## II. INTERMEDIATE CODE GENERATION

A source code can directly be translated into its target machine code, then why at all we need to translate the source code into an intermediate code which is then translated to its target code? Let us see the reasons why we need an intermediate code.



- If a compiler translates the source language to its target machine language without having the option for generating intermediate code, then for each new machine, a full native compiler is required.
- Intermediate code eliminates the need of a new full compiler for every unique machine by keeping the analysis portion same for all the compilers.
- The second part of compiler, synthesis, is changed according to the target machine.
- It becomes easier to apply the source code modifications to improve code performance by applying code optimization techniques on the intermediate code.

### Intermediate Representation

Intermediate codes can be represented in a variety of ways and they have their own benefits.

- **High Level IR** - High-level intermediate code representation is very close to the source language itself. They can be easily generated from the source code and we can easily apply code modifications to enhance performance. But for target machine optimization, it is less preferred.
- **Low Level IR** - This one is close to the target machine, which makes it suitable for register and memory allocation, instruction set selection, etc. It is good for machine-dependent optimizations.

### Why use an intermediate representation?

- It's easy to change the source or the target language by adapting only the front-end or back-end (**portability**)
- It makes **optimization** easier: one needs to write optimization methods only for the intermediate representation.
- The intermediate representation can be directly interpreted.

### How to choose the intermediate representation?

- It should be easy to translate the source language to the intermediate representation.
- It should be easy to translate the intermediate representation to the machine code.
- The intermediate representation should be suitable for optimization.
- It should be neither too high level nor too low level.
- One can have more than one intermediate representation in a single compiler.

## INTERMEDIATE LANGUAGES

**General forms of Intermediate Representations (IR)**

1. Graphical IR (parse tree, abstract syntax trees, DAG)
2. Linear IR (POSTFIX)
3. Three Address Code (TAC) - instructions of the form “**result = op1 operator op2**”

**Three Address Code Statements**

- Three Address code is a sequence of statements of the general form  $x = y \text{ op } z$ , where  $x$ ,  $y$  and  $z$  are names, constants and  $op$  stands for operator.
- In TAC, no multiple arithmetic expressions are permitted. Each statement contains almost three addresses, two for operands and one for the result.
- For example, the statement  $a = b + c * d$  become

$$t1 = c * d$$

$$t2 = b + t1$$

$$a = t2$$
**Types of TAC Statements**

| Type                                     | General form                                                                                                                                                                  |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Assignment statement                     | $x = y \text{ op } z$ , where $op$ – arithmetic / logical operator                                                                                                            |
| Unary assignment                         | $x := op \ y$ , where $op$ is any unary operator                                                                                                                              |
| Copy statement                           | $x = y$                                                                                                                                                                       |
| Unconditional Jump                       | goto L, where L is a label                                                                                                                                                    |
| Conditional Jump                         | if $x \text{ relop } y$ goto L, where $rel$ is any relational operator & L is a label                                                                                         |
| Function call / procedure call statement | For $p(a_1, a_2, \dots, a_n)$ ;<br>param $a_1$<br>param $a_2$<br>...<br>param $a_n$<br>call $p, n$<br><br>where $a_i$ - argument, $p$ – function name, $n$ – no. of arguments |
| Indexed statement                        | $x = y[i]$ and $x[i] = y$                                                                                                                                                     |

|                                           |                                   |
|-------------------------------------------|-----------------------------------|
| Address & Pointer<br>Assignment statement | $x = \&y$ , $x = *y$ and $*x = y$ |
|-------------------------------------------|-----------------------------------|

### Implementation of Three Address Code (TAC) Statements

The TAC statements are implemented as RECORD structure with fields as arguments and operators.

There are three kinds of representations as

1. Quadruples
2. Triples
3. Indirect Triples

### Quadruples

A Quadruple is a record structure with four fields as

| op | arg1 | arg2 | result |
|----|------|------|--------|
|----|------|------|--------|

- The contents of the fields arg1, arg2 and result are normally a pointer to the Symbol Table entries for the names represented by these fields.
- Drawbacks: Entering compiler generated temporaries into the symbol table requires additional memory, which leads to high space complexity.
- Example: The TAC sequence for the statement  $a = b + c * d$

$t1 = c * d$   
 $t2 = b + t1$   
 $a = t2$

| op     | arg1 | arg2 | result |
|--------|------|------|--------|
| *      | c    | d    | t1     |
| +      | b    | t1   | t2     |
| ASSIGN | t2   | --   | a      |

### Triples

To avoid entering the compiler generated temporaries into the symbol table, a temporary value is referred by the position of the statement that computes it.

In triples the TAC statements can be represented as record with three fields: op, arg1 and arg2.

| op | arg1 | arg2 |
|----|------|------|
|----|------|------|

Example: The TAC sequence for the statement  $a = b + c * d$

1.  $t1 = c * d$
2.  $t2 = b + t1$
3.  $a = t2$

| Statement / Position | op     | arg1 | arg2 |
|----------------------|--------|------|------|
| 1                    | *      | c    | d    |
| 2                    | +      | b    | [1]  |
| 3                    | ASSIGN | a    | [2]  |

In triples, the array references requires two entries,

Triples for statement  $x[i] = y$  which generates two records is as follows

| Statement / Position | op     | arg1 | arg2 |
|----------------------|--------|------|------|
| 0                    | []=    | x    | i    |
| 1                    | ASSIGN | [0]  | y    |

**Similarly** Triples for statement  $y = x[i]$  which generates two records is as follows

| Statement / Position | op     | arg1 | arg2 |
|----------------------|--------|------|------|
| 0                    | =[]    | x    | i    |
| 1                    | ASSIGN | y    | [0]  |

**Indirect Triples**

In Indirect Triples, list the pointer to triples rather than listing the triples themselves.

Example: The TAC sequence for the statement  $a = b + c * d$

1.  $t1 = c * d$
2.  $t2 = b + t1$
3.  $a = t2$

| Statement Position | Pointer |   | Address | op     | arg1 | arg2 |
|--------------------|---------|---|---------|--------|------|------|
| 1                  | 100     | → | 100     | *      | c    | d    |
| 2                  | 103     | → | 103     | +      | b    | [1]  |
| 3                  | 106     | → | 106     | ASSIGN | a    | [2]  |

**Comparison of TAC Statements**

1. By Storage Requirement: Both Quadruples and Indirect Triples require the same amount of memory, but using triples can save memory.
2. Support for Code Optimization: Quadruples find best usage in an optimizing compiler where statements are freely moved around. If the statement computing  $x$  is moved, the statement using the value of  $x$  requires no change.

A statement can be moved by recording the statement list in Indirect Triples, whereas in Triples moving a statement requires changes in all references to that temporary.

**SEMANTIC ANALYSIS**

- **Semantic analysis**, also context sensitive **analysis**, is a process in compiler construction, usually after parsing, to gather necessary **semantic** information from the source code.
- It usually includes type checking, scope resolution, array index bound checking or makes sure a variable is declared before use which is impossible to detect in parsing.
- Semantics of a language provide meaning to its constructs, like tokens and syntax structure. Semantics help interpret symbols, their types, and their relations with each other.
- Semantic analysis judges whether the syntax structure constructed in the source program derives any meaning or not.
- Lexical Analysis and Syntactic Analysis produce a “context free” analysis of the input, whereas the Semantic Analysis performs context sensitive checks.

**CFG + semantic rules = Syntax Directed Definitions**

**Semantic Error:** Some of the semantic errors that the semantic analyzer is expected to recognize:

1. Type mismatch
2. Undeclared variable
3. Reserved identifier misuse.
4. Multiple declaration of variable in a scope.
5. Accessing an out of scope variable.
6. Actual and formal parameter mismatch.

### **Attribute Grammar**

- Attribute grammar is a special form of context-free grammar where some additional information (attributes) are appended to one or more of its non-terminals in order to provide context-sensitive information.
- Each attribute has well-defined domain of values, such as integer, float, character, string, and expressions.
- Attribute grammar is a medium to provide semantics to the context-free grammar and it can help specify the syntax and semantics of a programming language.
- Attribute grammar (when viewed as a parse-tree) can pass values or information among the nodes of a tree.

**There are two notations used for associating semantic rules with productions,**

1. **SDD – Syntax Directed Definition**
2. **SDT – Syntax Directed Translation**

**SDD – Syntax Directed Definition:** A SDD is a generalization of CFG in which each grammar symbol has an associated set of attributes partitioned into two subsets called the synthesized attributes and inherited attributes of the grammar symbol.

**SDT – Syntax Directed Translation:** A SDT is a notation used to attach the semantic action to the production rules of the grammar.

### **SDT for Declaration statement**

- For every declaration, it is necessary to lay out storage for the declared variables.
- For every local name in a procedure, needs to create a ST(Symbol Table) entry containing:
  - The type of the name
  - How much storage the name requires
  - A relative offset from the beginning of the static data area or beginning of the activation record.
- To keep track of the current offset into the static data area, the compiler maintains a global variable called OFFSET.
- OFFSET is initialized to 0 when compilation begins.
- After each declaration, OFFSET is incremented by the size of the declared variable

### **Variable:**

**Offset** - a variable that keep track of next available relative address.

**Attributes:**

**T. Type** – defines the type of a data object, like integer, float, etc.

**T. width** – defines the amount of memory units taken by the data object.

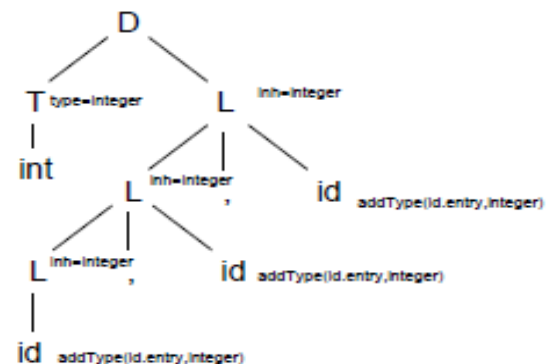
**Routines:**

**Enter (name, type, offset)** - This procedure should create an entry in the symbol table, for Variable *name*, having its type set to type and relative address *offset* in its data area.

| Grammar                        | Semantic Rules                                                |
|--------------------------------|---------------------------------------------------------------|
| $D \rightarrow T L$            | { L.type = T.type; Offset = 0; }                              |
| $T \rightarrow \text{int}$     | { T.type = integer; T.width = 2; offset = offset + T.width; } |
| $T \rightarrow \text{float}$   | { T.type = float; T.width = 4; offset = offset + T.width; }   |
| $L \rightarrow L_1, \text{id}$ | { L1.type = L.type; enter(id.name, L.type, offset) }          |
| $L \rightarrow \text{id}$      | { enter(id.name, L.type, offset) }                            |

**Example:** int a, b, c;

Symbol Table Entry ..



| Name | type    | width | offset | ... |
|------|---------|-------|--------|-----|
| ...  | ...     | ...   | ...    | ... |
| a    | integer | 2     | 0      | ... |
| b    | Integer | 2     | 2      | ... |
| C    | Integer | 2     | 4      | ... |
| ...  | ...     | ...   | ...    | ... |

**SDT for Assignment Statement**

- The Syntax Directed Translation to generate the TAC statements for the an assignment statement includes the following attributes and routines,

**Attributes:**

E. Place – refers to the address that will hold the value of E

**Routines:**

Emit () – routine to generate the TAC statement to an output file. For example the notation emit(x '=' y '+' z) represent the TAC instruction x=y+z. Here the expression appearing in place of variables like x,y and z are evaluated when passed to emit and quoted strings like '=' are taken literally.

Look-up (name) – This routine checks is there is an entry for the occurrence of the name in the symbol table. If so, a pointer to the entry is returned, otherwise returns NULL.

NewTemp() – routine to create a new compiler-generated temporary.

| Grammar                   | Semantic Rules                                                                                    |
|---------------------------|---------------------------------------------------------------------------------------------------|
| $S \rightarrow id = E$    | {P = LOOKUP (id.name)<br>If P ≠ NULL then EMIT( P '=' E.place);<br>else ERROR; }                  |
| $E \rightarrow E_1 + E_2$ | { E. Place = NEWTEMP();<br>EMIT ( E.place '=' E <sub>1</sub> .place '+' E <sub>2</sub> .place); } |
| $E \rightarrow E_1 * E_2$ | { E. Place = NEWTEMP();<br>EMIT ( E.place '=' E <sub>1</sub> .place '*' E <sub>2</sub> .place); } |
| $E \rightarrow - E_1$     | { E. Place = NEWTEMP();<br>EMIT ( E.place '=' 'UMINUS' E <sub>1</sub> .place); }                  |
| $E \rightarrow ( E_1 )$   | { E. Place = E <sub>1</sub> .place ; }                                                            |
| $E \rightarrow id$        | {P = LOOKUP (id.name)<br>If P ≠ NULL then E.place = P;<br>else ERROR; }                           |
| $E \rightarrow num$       | {E.place = num.val; }                                                                             |

Example: please refer Class work Note-book.

### SDT for Array References

The elements of an array are stored in a block of consecutive locations.

Let

**A** be a Single Dimensional Array, **W** be the width of an array element

The  $i^{\text{th}}$  entry of A is in location

$$A[i] = \text{base} + (i - \text{low}) * w$$

Where low – lower bound on the array subscript

Base – base address of the array (ie) relative address of A [low].

The same expression can be rewritten as

$$A[i] = i * w + C_A \quad \text{where } C_A = (\text{base} - \text{low} * w)$$

The Sub-expression  $C_A$  can be evaluated when the declaration of an array is seen and the value of  $C_A$  is kept available in the symbol table entry for A. Hence the relative address for  $A[i]$  is obtained by simple adding  $i*w$  to  $C_A$ .

Example: For the assignment  $B[i] = 100$ , in TAC form,

$$t_1 = i * w$$

$$C_B[t_1] = 100$$

| Grammar                                                       | Semantic Rules                                                                                                                              |
|---------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| $S \rightarrow L = E$                                         | <pre> { if L.offset= NULL     Emit(L.place '=' E.place );   else     Emit(L.place '[' L.offset ']' '=' E.place); }</pre>                    |
| $L \rightarrow \text{id} [\text{num}]$<br>$\quad   \text{id}$ | <pre> { L.place=NEWTEMP();   L.offset=NEWTEMP();   Emit(L.Place '=' C_id.arrayname );   Emit(L.offset='num*val '*' WIDTH(id.type)); }</pre> |
| $E \rightarrow L$                                             | <pre> { E.place=L.place;   Emit( E.place='L.place '[' L.offset ']' ); }</pre>                                                               |
| $E \rightarrow E_1 + E_2$                                     | <pre> { E. Place = NEWTEMP();   EMIT ( E.place '=' E1.place '+' E2.place); }</pre>                                                          |
| $E \rightarrow E_1 * E_2$                                     | <pre> { E. Place = NEWTEMP();   EMIT ( E.place '=' E1.place '*' E2.place); }</pre>                                                          |
| $E \rightarrow - E_1$                                         | <pre> { E. Place = NEWTEMP();</pre>                                                                                                         |

|                         |                                                                         |
|-------------------------|-------------------------------------------------------------------------|
|                         | EMIT ( E.place '=' 'UMINUS' E <sub>1</sub> .place); }                   |
| $E \rightarrow ( E_1 )$ | { E. Place = E <sub>1</sub> .place ; }                                  |
| $E \rightarrow id$      | {P = LOOKUP (id.name)<br>If P ≠ NULL then E.place = P;<br>else ERROR; } |
| $E \rightarrow num$     | {E.place = num.val; }                                                   |

**Example:** TAC sequence for  $a[i] = b * c + c * d$  as per the above semantic rules will be,

$t_1 = b * c$

$t_2 = c * d$

$t_3 = t_1 + t_2$

$t_4 = i * w$

$C_a[t_4] = t_3$

### SDT for Multi-dimensional Array References

The Two-dimensional array is normally stored in one of the two forms,

1. Row-major ordering
2. Column-major ordering

|           |           |           |
|-----------|-----------|-----------|
| A [1] [1] | A [1] [2] | A [1] [3] |
| A [2] [1] | A [2] [2] | A [2] [3] |

Row-Major ordering of an array A [2] [3]

|           |
|-----------|
| A [1] [1] |
| A [1] [2] |
| A [1] [3] |
| A [2] [1] |
| A [2] [2] |
| A [2] [3] |

|           |           |           |
|-----------|-----------|-----------|
| A [1] [1] | A [1] [2] | A [1] [3] |
| A [2] [1] | A [2] [2] | A [2] [3] |

Column-Major ordering of an array A [2] [3]

|           |
|-----------|
| A [1] [1] |
| A [2] [1] |
| A [1] [2] |
| A [2] [2] |
| A [1] [3] |
| A [2] [3] |

The Relative address of A [i] [j] can be calculated as

$A[i][j] = \text{base} + [(i - \text{low1}) * n2 + (j - \text{low2})] * w$ , for Row-major ordering

whereas

$$A[i][j] = \text{base} + [(j - \text{low2}) * n1 + (i - \text{low1})] * w, \text{ for Column-major ordering}$$

Where

low1 & low2 are the lower bounds on the array subscript

Base – base address of the array (ie) relative address of A [low1] [low2].

n1 & n2 – No of elements in dimension 1 and 2 respectively.

The Compile-time pre-calculation is also being applied to address calculation of multi-dimensional arrays. Hence the same expression can be rewritten as

$$A[i][j] = (((i * n2) + j) * w) + C_A \quad \text{where } C_A = \text{base} - (((\text{low1} * n2) + \text{low2}) * w)$$

The Sub-expression  $C_A$  can be evaluated when the declaration of an array is seen and the value of  $C_A$  is kept available in the symbol table entry for A. Hence the relative address for  $A[i][j]$  is obtained by adding  $t3$  to  $C_A$  as

$$t1 = i * n2$$

$$t2 = t1 + j$$

$$t3 = t2 * w$$

Example: For the assignment  $B[i][j] = 100$ , in TAC form,

$$t1 = i * n2$$

$$t2 = t1 + j$$

$$t3 = t2 * w$$

$$C_B[t3] = 100$$

The Syntax Directed Translation to generate the TAC statements for the an array reference within an assignment statement includes the following attributes and routines,

#### Attributes:

L. place – refers to the address that will hold the value of L

L. offset – If L-value is a simple name (id) then L.offset is NULL. If it is an array reference it contains the relative address of the array.

EListL. array – represents an array & a pointer to the symbol table entry for that array.

EListL.ndim – records the number of the dimensions in the array.

EListL. place – refers to the address that will hold the value of EList

#### Routines:

Emit () – routine to generate the TAC statement to an output file.

Limit (array,j) – returns  $N_j$  – the size of the  $j^{\text{th}}$  dimension of the array.

Width (array) – returns the width of an array element.

The Grammar is generalized to support multi-dimensional array as  $A[n_1, n_2, n_3, \dots, n_k]$

| Grammar               | Semantic Rules                                                                                             |
|-----------------------|------------------------------------------------------------------------------------------------------------|
| $S \rightarrow L = E$ | { if L.offset= NULL then Emit(L.place '=' E.place );<br>else Emit(L.place '[' L.offset ']' '=' E.place); } |

|                                 |                                                                                                                                                                                                                                               |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $E \rightarrow L$               | { if L.offset = NULL then E.place=L.place;<br>else<br>E.place = NEWTEMP();<br>Emit( E.place='L.place '['L.offset']'); }                                                                                                                       |
| $L \rightarrow EList ]$         | { L.place=NEWTEMP();<br>L.offset= NEWTEMP();<br>Emit(L.place '=' C(EList.array));<br>Emit(L.offset=' EList.place '*' Width(EList.array));}                                                                                                    |
| $EList \rightarrow EList_1 , E$ | t=NEWTEMP();<br>m= EList <sub>1</sub> .ndim +1;<br>Emit(t=' EList <sub>1</sub> .place '*' Limit(EList <sub>1</sub> .array,m));<br>Emit(t=' t '+' E.place);<br>EList.array = EList <sub>1</sub> . array;<br>EList.place = t; EList.ndim = m; } |
| $EList \rightarrow id[ E$       | { EList.array = id.place;<br>EList.place = E.place; EList.ndim = 1; }                                                                                                                                                                         |
| $E \rightarrow E_1 + E_2$       | { E. Place = NEWTEMP();<br>EMIT ( E.place '=' E <sub>1</sub> .place '+' E <sub>2</sub> .place); }                                                                                                                                             |
| $E \rightarrow E_1 * E_2$       | { E. Place = NEWTEMP();<br>EMIT ( E.place '=' E <sub>1</sub> .place '*' E <sub>2</sub> .place); }                                                                                                                                             |
| $E \rightarrow - E_1$           | { E. Place = NEWTEMP();<br>EMIT ( E.place '=' 'UMINUS' E <sub>1</sub> .place); }                                                                                                                                                              |
| $E \rightarrow ( E_1 )$         | { E. Place = E <sub>1</sub> .place ; }                                                                                                                                                                                                        |
| $E \rightarrow id$              | {P = LOOKUP (id.name)<br>If P ≠ NULL then E.place = P; else ERROR; }                                                                                                                                                                          |
| $E \rightarrow num$             | {E.place = num.val; }                                                                                                                                                                                                                         |

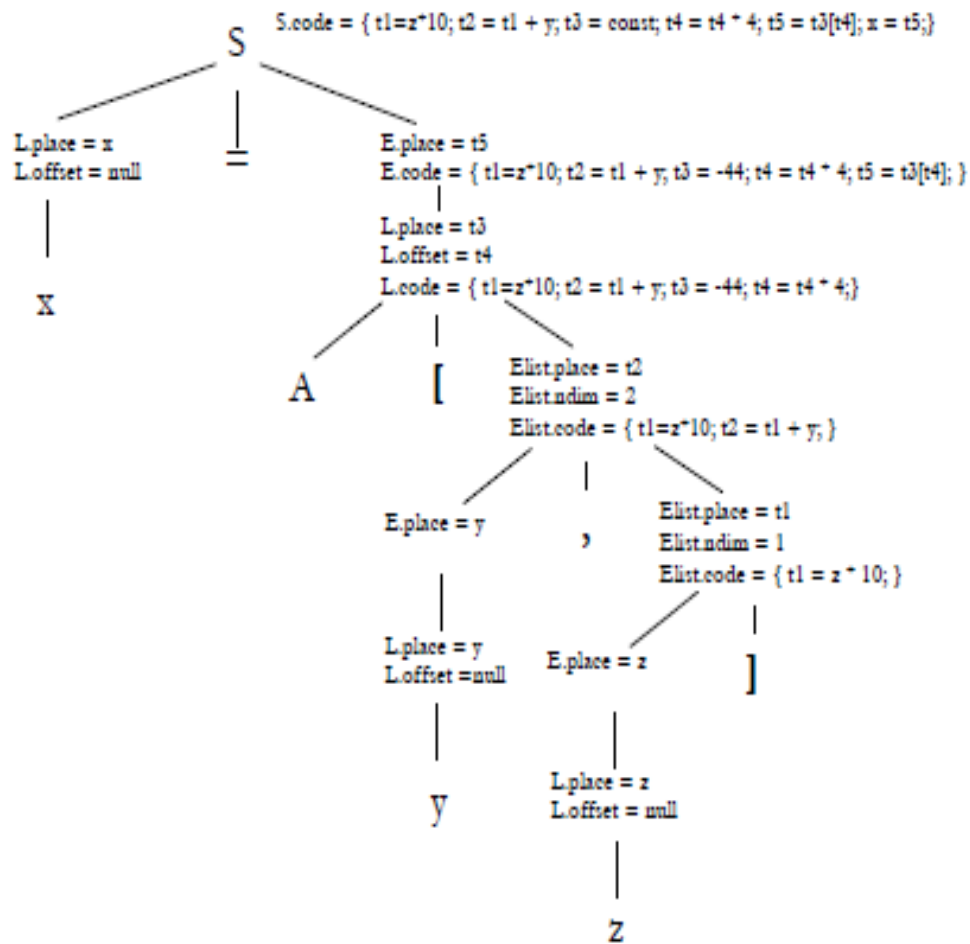
Similarly the TAC Instruction for 3 – dimensional arrays can be generated as

$$A[i] [j] [k] = (((i * n_2) + j) * n_3) * w + C_A$$

And the TAC Sequence will be,

$$\begin{aligned} t_1 &= i * n_2 \\ t_2 &= t_1 + j \\ t_3 &= t_2 * n_3 \end{aligned}$$

$t4 = t3 * w$   
 $C_A[t4]$



**SDT for SWITCH – CASE statement**

- In the Switch statement, a selector expression is to be evaluated followed by n constant values that the expression might take including a default value, which always matches if no other value does.

| Syntax                                                                                                                                                                                                                                                                                                                                                                                                                               | Semantic Rules                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><i>Switch (Expr) {</i></b><br><b><i>case v<sub>1</sub>:</i></b><br><b><i>S<sub>1</sub>;</i></b><br><b><i>break;</i></b><br><b><i>case v<sub>2</sub>:</i></b><br><b><i>S<sub>2</sub>;</i></b><br><b><i>break;</i></b><br>...<br><b><i>case v<sub>n-1</sub>:</i></b><br><b><i>S<sub>n-1</sub>;</i></b><br><b><i>break;</i></b><br><b><i>default:</i></b><br><b><i>S<sub>n</sub>;</i></b><br><b><i>break;</i></b><br><b><i>}</i></b> | <b>Code to evaluate E into 't'</b><br><b>goto TEST</b><br><b>L1: code for S<sub>1</sub></b><br><b>goto NEXT</b><br><b>L2: code for S<sub>2</sub></b><br><b>goto NEXT</b><br>...<br><b>Ln-1: code for S<sub>n-1</sub></b><br><b>goto NEXT</b><br><b>Ln: code for S<sub>n</sub></b><br><b>goto NEXT</b><br><b>TEST: if t=1 then goto L1</b><br><b>if t=2 then goto L2</b><br>...<br><b>if t=(n-1) then goto Ln-1</b><br><b>else goto Ln</b> |

**SDT for Procedure Call statement**

- The translation for procedure call statement includes a calling sequence and return sequence.
- Calling sequence- series of actions to be taken while entering into the procedure definition.
- Return sequence - series of actions to be taken while leaving from the procedure definition

| Grammar                                        | Semantic Rules                                                              |
|------------------------------------------------|-----------------------------------------------------------------------------|
| $D \rightarrow T \text{ id } (F) \{ S \}$      | { For each item P on QUEUE<br>Emit( 'PARAM' P);<br>Emit (Call id.name, n);} |
| $F \rightarrow T \text{ id, } F \mid \epsilon$ | {Append id.place to the end of the QUEUE; }                                 |

Example: Generate the TAC instruction for  $n = f(a[i]);$

```

t1 = i * w
t2 = a[t1]
PARAM t2
t3 = CALL f,1
n = t3

```

**SDT for BOOLEAN EXPRESSIONS**

- Boolean expressions are composed of the Boolean operators (*and*, *or* and *not*) applied to the elements that are Boolean variables or relational expressions.
- Example: **a or b and not c**

```

t1 = not c
t2 = b and t1
t3 = a or t2

```
- There are two methods of translating Boolean Expression,
  1. Numerical Representation
  2. Flow-of-control representation

**1. Numerical Representation:**

- Encode TRUE as 1 and FALSE as 0 and to evaluate a boolean expression.
- An expression can be evaluated completely from left to right.
- Routines:
  - NEWTEMP() – used to generate the new compiler-generated temporary.
  - EMIT() - used to generate a TAC Instruction
  - NEXTSTAT – Gives the index of the next TAC Instruction in the output sequence and every call to EMIT() increments NEXTSTAT by 1.

**Grammar:**  $E \rightarrow E \text{ or } E \mid E \text{ and } E \mid \text{not } E \mid (E) \mid \text{id}_1 \text{ relop id}_2 \mid \text{true} \mid \text{false}$

| Grammar                                | Semantic Rules                                                                                                                                                                |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $E \rightarrow E1 \text{ or } E2$      | { E.place := newtemp ( );<br>EMIT (E.place ':= ' E1.place 'or' E2.place); }                                                                                                   |
| $E \rightarrow E1 \text{ and } E2$     | { E.place := newtemp();<br>EMIT (E.place ':= ' E1.place 'and' E2.place); }                                                                                                    |
| $E \rightarrow \text{not } E$          | { E.place := newtemp();<br>EMIT ( E.place ':= ' 'not' E.place); }                                                                                                             |
| $E \rightarrow ( E1 )$                 | { E.place := E1.place; }                                                                                                                                                      |
| $E \rightarrow id1 \text{ relop } id2$ | { E.place := newtemp;<br>EMIT ('if' id1.place relop.op id2.place 'goto' nextstat+3);<br>EMIT (E.place ':= ' '0');<br>EMIT ('goto' nextstat+2);<br>EMIT (E.place ':= ' '1'); } |
| $E \rightarrow \text{true}$            | { E.place := newtemp;<br>EMIT (E.place ':= ' '1'); }                                                                                                                          |
| $E \rightarrow \text{false}$           | { E.place = newtemp;<br>emit (E.place ':= ' '0'); }                                                                                                                           |

**Example1:**  $a < b$  or  $c < d$  using numerical representation.

```

1000  if a<b goto 1003
1001  t1:=0
1002  goto 1004
1003  t1=1
1004  if c<d goto 1007
1005  t2=0
1006  goto 1008

```

**1007 t2=1**

**1008 t3 = t1 OR t2**

**Example2:** Translation of  $a < b$  or  $c < d$  and  $e < f$

```

100  if a<b goto 103
101  t1 := 0
102  goto 104
103  t1 := 1
104  if c<d goto 107
105  t2 := 0
106  goto 108

```

```

107   t2 := 1
108   if e<f goto 111

```

```

109   t3 := 0
110   goto 112

```

## 2. Flow-of-Control Representation:

- Control flow translation of Boolean expression representing the value of a Boolean expression by apposition reached in a program.

### Attributes:

- B.True – refers to the label to which control flows if B is true.
- B.False – refers to the label to which control flows if B is false.
- B.code – refers the sequence of TAC statements in B

### Routines:

- NewLabel () – returns a new label each time it is called.
- Gen() – attach the label to the TAC instruction.

| Grammar                                                  | Semantic Rules                                                                                                                          |
|----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b><math>E \rightarrow E1 \text{ or } E2</math></b>      | <pre> {E1.true = E.True E1.false = NewLabel(); E2.true = E.true E2.false=E.false E.code =E1.code    gen(E1.false)   E2.code }</pre>     |
| <b><math>E \rightarrow E1 \text{ and } E2</math></b>     | <pre> { E1.true =NewLabel(); E1.false = E.false E2.true = E.true E2.false = E.false E.code = E1.code    gen(E1.true)    E2.code }</pre> |
| <b><math>E \rightarrow \text{not } E1</math></b>         | <pre> { E.true = E1.false E.false = E1.true E.code = E1.code }</pre>                                                                    |
| <b><math>E \rightarrow id1 \text{ relop } id2</math></b> | <pre> {E.code = E1.code    E2.code    EMIT ('if' id1.place relop.op id2.place 'goto' E.true); EMIT ('goto' E.false);</pre>              |

|                       |                                   |
|-----------------------|-----------------------------------|
| $E \rightarrow true$  | { E.code = EMIT (goto E.true); }  |
| $E \rightarrow false$ | { E.code = EMIT (goto E.false); } |

Example:  $a < b$  or  $c < d$  using flow of control representation

```

        if a<b goto Ltrue
        goto L1
L1:     if c<d goto Ltrue
        goto Exit
Ltrue:

```

Example2: Translation of  $a < b$  or  $c < d$  and  $e < f$

```

        if a<b goto L1
        goto L2
L2:     if c<d goto L1
        goto Exit
L1:     if e<f goto Ltrue
        goto Exit

```

### SDT for Flow of Control Statements

The translation of Boolean expressions into TAC in the context of statement are generated by

$$S \rightarrow \text{if } ( B ) S_1$$

$$S \rightarrow \text{if } ( B ) S_1 \text{ else } S_2$$

$$S \rightarrow \text{while } ( B ) S_1$$

In this grammar, the non-terminal B represents a Boolean expression and S represents a statement.

S.Code – gives the translation into TAC instructions.

S.next – denoting a label for the instruction immediately after the code for S

NewLabel () – creates a new Label each time it is called.

Gen(L) – attaches the label L to the next TAC Instruction to be generated.

| Grammar                                                | Semantic Rules                                                                                                                                                                                                                               |
|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $S \rightarrow \text{if } ( B ) S_1$                   | B.true = NewLabel();<br>B.false = S <sub>1</sub> .next = S.next;<br>S.code = B.code    gen(B.true':')    S <sub>1</sub> .code                                                                                                                |
| $S \rightarrow \text{if } ( B ) S_1 \text{ else } S_2$ | B.true = NewLabel();<br>B.false = NewLabel ();<br>S <sub>1</sub> .next = S <sub>2</sub> .next = S.next;<br>S.code = B.code    gen(B.true':')    S <sub>1</sub> .code   <br>gen('goto' S.next)   <br>gen(B.false ':')    S <sub>2</sub> .code |
| $S \rightarrow \text{while } ( B ) S_1$                | begin =NewLabel ();<br>B.true = NewLabel();<br>B.false = S.next<br>S <sub>1</sub> .next = begin<br>S.code = gen (begin ':')    B.code    gen(B.true ':')   <br>S <sub>1</sub> .code    gen ('goto' begin)                                    |

### BACKPATCHING

- A key problem when generating code for Boolean expression and flow-of-control statements is that of matching a jump instruction with the target of the jump.
- TAC for Boolean expressions and flow-of-control statements are generated using two passes
  - Pass 1 – Generate the TAC instruction with the target of the jumps are unspecified.
  - Pass 2 – Identifying and filling up the labels at appropriate jump statements.

- To generate TAC instructions in a single pass, a compiler may not know the labels that control must goto at the time of goto statements are generated.
- Backpatching can be used to generate code for Boolean expressions and flow-of-control statements in one pass.
- It maintains a list of TAC statements that needed to be completed with the same label. All of the jumps on a list have the same target label.

**Attributes:**

B.trueList – will be list of jump instructions into which control goes if B is true.

B.falseList – will be list of jump instructions into which control goes if B is false.

S.nextList – denoting a list of jumps to the instruction immediately following the code for S

**Routines:**

Makelist (i) – creates a new list containing only i.

Merge (p1,p2) – Concatenates the lists pointed to by p1 with p2

Backpatch (p,i) – inserts i as target label for each of the instruction on the list pointed to by p.

**Backpatching for Flow-of-Control Statements:**

| Grammar                                                          | Semantic Rules                                                                                                                                                  |
|------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $S \rightarrow \text{if } ( B ) M S_1$                           | { Backpatch (B.trueList, M.Instr);<br>S.nextList = merge (B.falseList, S1.nextList); }                                                                          |
| $S \rightarrow \text{if } ( B ) M_1 S_1 N \text{ else } M_2 S_2$ | { Backpatch (B.trueList, M1.Instr);<br>Backpatch ( B.falseList, M2.Instr);<br>Temp=merge(S1.nextList, N.nextList);<br>S.nextList = merge (temp, S2.nextList); } |
| $S \rightarrow \text{while } M_1 ( B ) M_2 S_1$                  | { Backpatch (S1.nextList, M1.Instr);<br>Backpatch (B.trueList, M2.Instr);<br>S.nextList = B.falseList;<br>Gen ('goto' M1.Instr); }                              |
| $M \rightarrow \epsilon$                                         | {M.Instr = nextInstr; }                                                                                                                                         |
| $N \rightarrow \epsilon$                                         | {N.nextList = makelist (nextInstr);<br>Gen ('goto -'); }                                                                                                        |

Backpatching for Boolean Expressions:

| Grammar                                | Semantic Rules                                                                                                                                       |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| $B \rightarrow B_1 \mid \mid M B_2$    | { Backpatch (B1.falseList, M.Instr);<br>B.trueList = merge (B1.trueList, B2.trueList);<br>B. falseList = B2.falseList; }                             |
| $B \rightarrow B_1 \&\& M B_2$         | { Backpatch (B1.trueList,M.Instr);<br>B.trueList = B2.trueList;<br>B.falseList = merge (B1.falseList, B2.falseList);}                                |
| $B \rightarrow ! B_1$                  | { B. trueList = B1.falseList;<br>B.falseList = B1.trueList; }                                                                                        |
| $B \rightarrow E_1 \text{ relop } E_2$ | {B.trueList =makelist (nextInstr);<br>B.falseList = makelist( nextInstr +1);<br>Gen ('if' E1.place relop.op E2.place 'goto _');<br>Gen ('goto _'); } |
| $B \rightarrow \text{true}$            | { B.trueList = makeList (nextInstr);<br>Gen ('goto _'); }                                                                                            |
| $B \rightarrow \text{false}$           | { B.falseList = makeList (nextInstr);<br>Gen ('goto _'); }                                                                                           |
| $M \rightarrow \epsilon$               | {M.Instr = nextInstr; }                                                                                                                              |

### III. ERROR DETECTION AND RECOVERY

- A compiler is expected to assist the programmer in locating and tracking down errors that inevitably creep into the programs, despite the programmer's efforts.
- Common programming **errors can occur at different levels**,
  1. **Lexical Errors** – include misspelling of identifiers, keywords, or operators.
  2. **Syntactic Errors** – any violations in the syntax rule of the language
  3. **Semantic Errors** – includes type mismatches between operators and operands.
  4. **Logical Errors** – can be anything from incorrect reasoning on the part of the programmer.
- **Goals of Error handler in a parser**
  1. Report the presence of errors clearly and accurately.
  2. Recover from each error quickly enough to detect subsequent errors.
  3. Add minimal overhead to the processing of correct programs.

#### ERROR RECOVERY STRATEGIES

- Once an error is detected, the parser should recover quickly to detect subsequent errors in the source program.
- Some of the familiar **Error recovery strategies are: Panic mode, Phrase level, Error productions and global corrections.**

#### Error Recovery Strategies employed in a Parser

##### (1) Panic Mode Recovery

- On discovering an error, the parser discards input symbols one at a time until one of a designated set of synchronizing tokens is found.
- The synchronizing tokens are usually delimiters such as ; or }.
- The panic-mode correction often skips a considerable amount of input without checking it for additional errors.
- Advantages:
  - Simplicity
  - Guaranteed not to go into an infinite loop while processing.

##### (2) Phrase Level Recovery

- On discovering an error, a parser may perform local corrections on the remaining input, i.e it may replace a prefix of remaining input by some string that allows the parser to continue.
- The choice of local correction is left to the compiler designers.
- Drawback: the difficulty it has in coping with situations in which the actual error has occurred before the point of detection.

**(3) Error Productions**

- By anticipating common errors that might be occurred, the compiler designer can augment the grammar for the language with productions that generate the erroneous constructs.
- A parser constructed from a grammar augmented by these error productions detects the anticipated errors when an error production is used during parsing.
- The parser can then generate appropriate error diagnostics about the erroneous construct that has been recognized in the input.

**(4) Global Corrections**

- Ideally any compiler would like to make as few changes as possible in processing an incorrect input string.
- There are algorithms for choosing a minimal sequence of changes to obtain a globally least-cost correction.
- Given an incorrect input string X and grammar G, these algorithms will find a parse tree for a related string Y, such that the modification required to transform X into Y is as small as possible.

Drawback: Global correction algorithms are too costly to implement in terms of time and space.